Aim:

To implement a Binary Search Tree (BST) using linked nodes in C and perform insertion, inorder, preorder, postorder traversal, and searching operations.

Algorithm:

Algorithm: Node creation

1. Define a structure Node containing:
    - data
    - pointer to the left child
    - pointer to the right child.
2. Create a function createNode() to allocate memory for a new node.
3. Store the given value in the node.
4. Initialize the left and right pointers to NULL.

Algorithm: Insertion

1. Start with the root node.
2. If the root is NULL, create a new node and make it the root.
3. Compare the new value with the current node's value.
4. If the new value is **smaller**, move to the left subtree.
5. If the new value is **greater**, move to the right subtree.
6. Repeat the comparison until an empty position is found.
7. Insert the new node at that position.
8. If the value is already present, do not insert it.

Algorithm: Searching

1. Start from the root.
2. Compare the search value with the current node.
3. If the current node is NULL, the element is not present.
4. If the search value is equal to the current node's value, the element is found.
5. If the search value is smaller, search the left subtree.
6. If the search value is greater, search the right subtree.
7. Continue until the element is found or the subtree becomes empty.

Algorithm: Inorder Traversal

Inorder follows:

LEFT → ROOT → RIGHT

1. Check whether the root is NULL.
2. If not NULL, traverse the left subtree.
3. Visit and print the root.
4. Traverse the right subtree.

Algorithm: Preorder Traversal

Preorder follows:

ROOT → LEFT → RIGHT

Algorithm

1. Check whether the root is NULL.
2. Print the root.
3. Traverse the left subtree.
4. Traverse the right subtree.

Algorithm: Postorder Traversal

Postorder follows:

LEFT → RIGHT → ROOT

Algorithm

1. Check whether the root is NULL.
2. Traverse the left subtree.
3. Traverse the right subtree.
4. Print the root.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
// Define the structure for a BST node
struct Node {
   int data;
   struct Node* left;
   struct Node* right;
};
// Function to create a new node
struct Node* createNode(int value) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   if (newNode == NULL) {
      printf("Memory allocation failed!\n");
      exit(1);
   }
   newNode->data = value;
   newNode->left = NULL;
   newNode->right = NULL;
   return newNode;
}
// Function to insert a value into the BST
struct Node* insert(struct Node* root, int value) {
   // If the tree/subtree is empty, return a new node
   if (root == NULL) {
      return createNode(value);
   }

   // Otherwise, recur down the tree
   if (value < root->data) {
      root->left = insert(root->left, value);
   } else if (value > root->data) {
      root->right = insert(root->right, value);
   } else {
      printf("Duplicate value %d ignored.\n", value);
   }
   return root;
}

// Function to search for a value in the BST
struct Node* search(struct Node* root, int key) {
   // Base Cases: root is null or key is present at root
   if (root == NULL || root->data == key) {
      return root;
   }
```

```
    // Key is smaller than root's data
    if (key < root->data) {
        return search(root->left, key);
    }

    // Key is greater than root's data
    return search(root->right, key);
}

// Inorder traversal (Left, Root, Right) - Prints in sorted order
void inorder(struct Node* root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

// Preorder traversal (Root, Left, Right)
void preorder(struct Node* root) {
    if (root != NULL) {
        printf("%d ", root->data);
        preorder(root->left);
        preorder(root->right);
    }
}

// Postorder traversal (Left, Right, Root)
void postorder(struct Node* root) {
    if (root != NULL) {
        postorder(root->left);
        postorder(root->right);
        printf("%d ", root->data);
    }
}

// Main function providing a menu-driven interface
int main() {
    struct Node* root = NULL;
    int choice, value, key;
    root = insert(root, 50);
    root = insert(root, 10);
    root = insert(root, 30);
    root = insert(root, 70);
    root = insert(root, 80);

            printf("Inorder Traversal: ");
```

```
        inorder(root);
        printf("\n");
        printf("Preorder Traversal: ");
        preorder(root);
        printf("\n");
        printf("Postorder Traversal: ");
        postorder(root);

        printf("\n Enter value to search: ");
        scanf("%d", &key);
        if (search(root, key) != NULL) {
           printf(" \n Element %d found in the BST.\n", key);
        } else {
           printf("\n Element %d NOT found in the BST.\n", key);
        }

   return 0;
}
```

OUTPUT:

Inorder Traversal: 10 30 50 70 80

Preorder Traversal: 50 10 30 70 80

Postorder Traversal: 30 10 80 70 50

Enter value to search: 30

Element 30 found in the BST.

Inorder Traversal: 10 30 50 70 80

Preorder Traversal: 50 10 30 70 80

Postorder Traversal: 30 10 80 70 50

Enter value to search: 55

Element 55 NOT found in the BST.

Result

Thus, a Binary Search Tree was successfully implemented in C and insertion, tree traversals, and searching operations were performed successfully.